

Урок 1: Первая страница на React

Пошаговый план: от установки до GitHub Pages

Для преподавателя: готовый сценарий с командами и репликами.

Цель урока

К концу занятия у каждого ученика:

- работает локальный сайт на `localhost`
- создана страница из компонентов
- проект загружен в GitHub
- сайт опубликован в GitHub Pages

Пример результата: <https://vologjanka.github.io/bCard/>

Тайминг 3,5 часа

- 0:00-0:10 — старт и мотивация
- 0:10-0:25 — что такое сайт / React / компонент
- 0:25-1:05 — установка Cursor, Node.js, Git
- 1:05-1:35 — создание проекта и запуск
- 1:35-2:45 — компоненты + CSS Modules + сборка страницы
- 2:45-3:10 — Git и GitHub (init -> push)
- 3:10-3:30 — deploy на GitHub Pages

Слайд для старта (0:00-0:10)

Реплика преподавателя:

Сегодня вы сделаете свою первую страницу на React и опубликуете ее в интернете.

Если что-то не работает — это нормально. Ошибки подсказывают, что исправить.

Действия:

- показать готовый пример сайта
- коротко проговорить план урока

Шаг 1. Объяснить базу (0:10-0:25)

Реплики:

Сайт — это набор файлов HTML/CSS/JS, который показывает браузер.

React — это способ собирать страницу из блоков.

Блок называется компонент.

Схема на доске:

```
App -> FullName + ProfilePhoto + AboutMe
```

Шаг 2. Установить Cursor (0:25-0:35)

Что делаем:

1. Открыть <https://cursor.com>
2. Скачать версию для Windows
3. Установить и запустить

Для учителя:

- проверить, что у всех открылся редактор
- показать: файлы слева, код в центре, терминал снизу

Шаг 3. Установить Node.js (0:35-0:50)

Что делаем:

1. Открыть <https://nodejs.org>
2. Установить LTS (20+ или 22+)
3. Проверить:

```
node -v  
npm -v
```

Для учителя:

- если версия не выводится, перезапустить терминал

Шаг 4. Установить Git (0:50-1:05)

Что делаем:

1. Открыть <https://git-scm.com/download/win>
2. Установить с настройками по умолчанию
3. Проверить:

```
git --version
```

Для учителя:

- у всех должна выводиться версия Git

Шаг 5. Создать проект (1:05-1:20)

Команды:

```
cd C:\js\react\bCard  
npm create vite@latest my-app -- --template react-ts  
cd my-app  
npm install
```

Реплика:

Сейчас мы создали структуру проекта и установили все нужные библиотеки.

Шаг 6. Поднять фронт локально (1:20-1:35)

Команда:

```
npm run dev
```

Реплика:

Откройте адрес из терминала. Обычно это localhost:5173.

Для учителя:

- убедиться, что у всех открылась стартовая страница
- если порт занят, Vite покажет другой порт

Шаг 7. Объяснить ключевые файлы (1:35-1:45)

- `src` — основной код
- `public` — статические файлы (фото, иконки)
- `package.json` — паспорт проекта (команды и зависимости)

Реплика:

Если вы не в папке с `package.json`, npm-команды не работают.

Шаг 8. Настроить package.json для deploy

Добавить скрипты:

```
"scripts": {  
  "dev": "vite",  
  "build": "tsc -b && vite build",  
  "preview": "vite preview",  
  "predeploy": "npm run build",  
  "deploy": "gh-pages -d dist"  
}
```

Для учителя:

- пояснить разницу `dependencies` и `devDependencies`

Шаг 9. Создать компонент ProfilePhoto (1:45-2:05)

ProfilePhoto.tsx :

```
import styles from './ProfilePhoto.module.css'

function ProfilePhoto() {
  return (
    <section className={styles.photoCard} aria-label="Фото">
      <img
        className={styles.photo}
        src={` ${import.meta.env.BASE_URL}myfoto.jpg`}
        alt="Фото Дувановой Ирины"
      />
    </section>
  )
}

export default ProfilePhoto
```

Шаг 10. Подключить CSS модуль к фото

ProfilePhoto.module.css :

```
.photoCard { border: 2px solid #d8b4fe; border-radius: 12px; padding: 16px 20px; }  
.photo { width: min(100%, 420px); height: auto; display: block; border-radius: 12px; }
```

Для учителя:

- объяснить, что CSS Modules изолируют стили компонента

Шаг 11. Создать AboutMe и FullName (2:05-2:35)

Что делаем:

- `AboutMe.tsx` + `AboutMe.module.css`
- `FullName.tsx` + `FullName.module.css`

Реплика:

Каждый блок страницы — отдельный компонент со своими стилями.

Шаг 12. Собрать страницу в App (2:35-2:45)

App.tsx :

```
import './App.css'  
import AboutMe from './components/AboutMe'  
import FullName from './components/FullName'  
import ProfilePhoto from './components/ProfilePhoto'
```

Расположение: `FullName` сверху, `ProfilePhoto` и `AboutMe` в одной строке.

Шаг 13. Сделать макет и адаптив

App.css :

```
.page { display: flex; flex-direction: column; gap: 16px; }  
.content-row { display: flex; gap: 16px; }  
@media (max-width: 900px) { .content-row { flex-direction: column; } }
```

Для учителя:

- показать, как страница меняется на узком экране

Шаг 14. Создать репозиторий и отправить код (2:45-3:10)

Команды Git:

```
cd C:\js\react\bCard
git init
git add .
git commit -m "Initial commit"
git remote add origin https://github.com/<username>/<repo>.git
git branch -M main
git push -u origin main
```

Для учителя:

- проверить, что у ученика появилась ветка `main` на GitHub

Шаг 15. Настроить Vite для GitHub Pages

vite.config.ts :

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  base: './',
  plugins: [react()],
  server: { host: 'localhost', port: 3000, open: true }
})
```

Шаг 16. Deploy на GitHub Pages (3:10-3:30)

Команды:

```
cd C:\js\react\bCard\my-app  
npm install -D gh-pages  
npm run deploy
```

GitHub:

```
Settings -> Pages -> Branch gh-pages -> Folder / (root)
```

Быстрый блок “если не работает”

- `ENOENT package.json` -> перейти в папку `my-app`
- `src refsпес mai` -> опечатка, нужно `main`
- нет картинки на Pages -> использовать `import.meta.env.BASE_URL`
- не обновился сайт -> снова `npm run deploy` и обновить страницу с `Ctrl + F5`

Финальный чек-лист для учителя

- у всех запускается `npm run dev`
- у всех есть 3 компонента и CSS Modules
- у всех выполнен `git push origin main`
- у всех выполнен `npm run deploy`
- у всех открывается сайт по ссылке GitHub Pages

Конец урока

Реплика преподавателя:

Сегодня вы прошли полный цикл веб-разработки:
установка, код, стили, Git, публикация.

На следующем уроке добавим новые блоки и улучшим дизайн.